



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Chap 13. Component Level Design

- ❖ **1. The Concepts of Component**
- ❖ **2. Component Level Design**
- ❖ **3. Basic Design Principles**
 - **The Open-Closed Principle (OCP).**
 - **The Liskov Substitution Principle (LSP)**
 - **Dependency Inversion Principle (DIP).**
 - **The Interface Segregation Principle (ISP).**
 - **Etc.**
- ❖ **4. Cohesion & Coupling**
- ❖ **5. Component Level Design**

What is a Component?

❖ *OMG Unified Modeling Language Specification* [OMG01] defines a component as

- “... a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.”

“.....系统的一个模块化的、可部署的、可替换的部分，它封装了实现并暴露了一组接口。”

What is a Component?

- ❖ ***OO view:*** a component contains a set of collaborating classes
- ❖ ***Conventional view:*** a component contains processing logic, the internal data structures that are required to implement the processing logic, **and an interface** that enables the component to be invoked and data to be passed to it.

1. 什么是构件

一个模块化的、可部署的、可替换的系统部分，它封装了实现并公开了一组接口。——OMG
从OO角度来看——一系列相互协作的类

Component-level Design

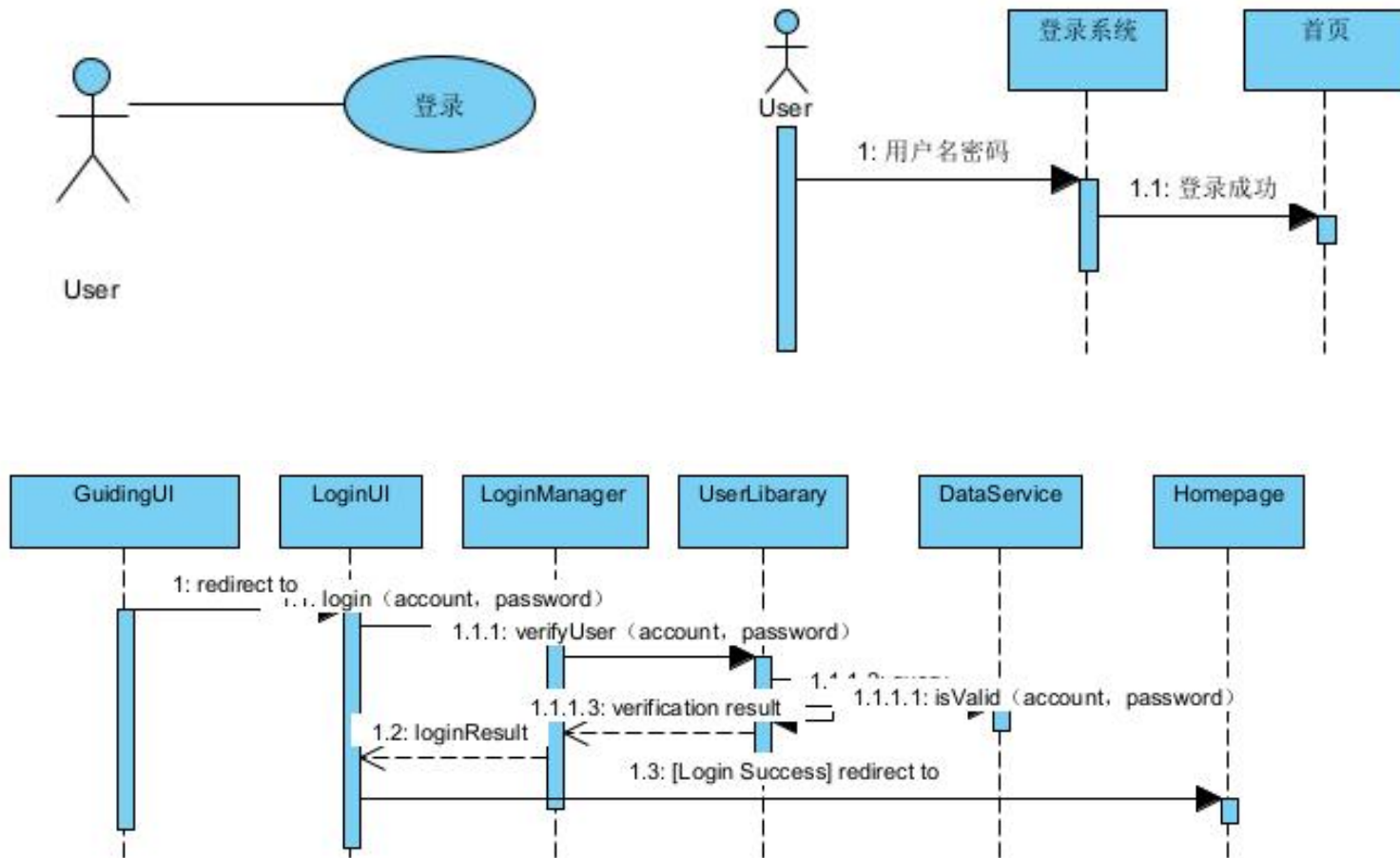
- ❖ **Component-level design defines the data structures, algorithms, interface characteristics, and communication mechanisms allocated to each component.**

2. 构件定义了

- 数据结构
- 算法
- 接口特征
- 通信机制

Component-level Design

❖ Design detailed solutions for use cases



Component-level Design

❖ Design detailed classes

- **Determine the visible range of the class.**
- **Refine the relationship between the classes.**
- **Refine the properties of the classes**
- **Refine the methods of the classes**
- **Review the design of the classes**
- 设计详细类
 - 确定类的可见范围。
 - 细化类之间的关系。
 - 细化类的属性。
 - 细化类的方法。
 - 审查类的设计。

Component-level Design

❖ Design data structure

- 1. Determine the data to be persisted
- 2. Determine the way data is stored and organized
- 3. Design data operations
- 4. Review

- 设计数据结构

1. 确定要持久化的数据
2. 确定数据的存储和组织方式
3. 设计数据操作
4. 审查

Basic Design Principles

- ❖ **The Open-Closed Principle (OCP).** *“A module [component] should be open for extension but closed for modification.”*
- ❖ **The Liskov Substitution Principle (LSP).** *“Subclasses should be substitutable for their base classes.”*
- ❖ **Dependency Inversion Principle (DIP).** *“Depend on abstractions. Do not depend on concretions.”*
- ❖ **The Interface Segregation Principle (ISP).** *“Many client-specific interfaces are better than one general purpose interface.”*
- ❖ **The Release Reuse Equivalency Principle (REP).** *“The granule of reuse is the granule of release.”*
- ❖ **The Common Closure Principle (CCP).** *“Classes that change together belong together.”*
- ❖ **The Common Reuse Principle (CRP).** *“Classes that aren’t reused together should not be grouped together.”*

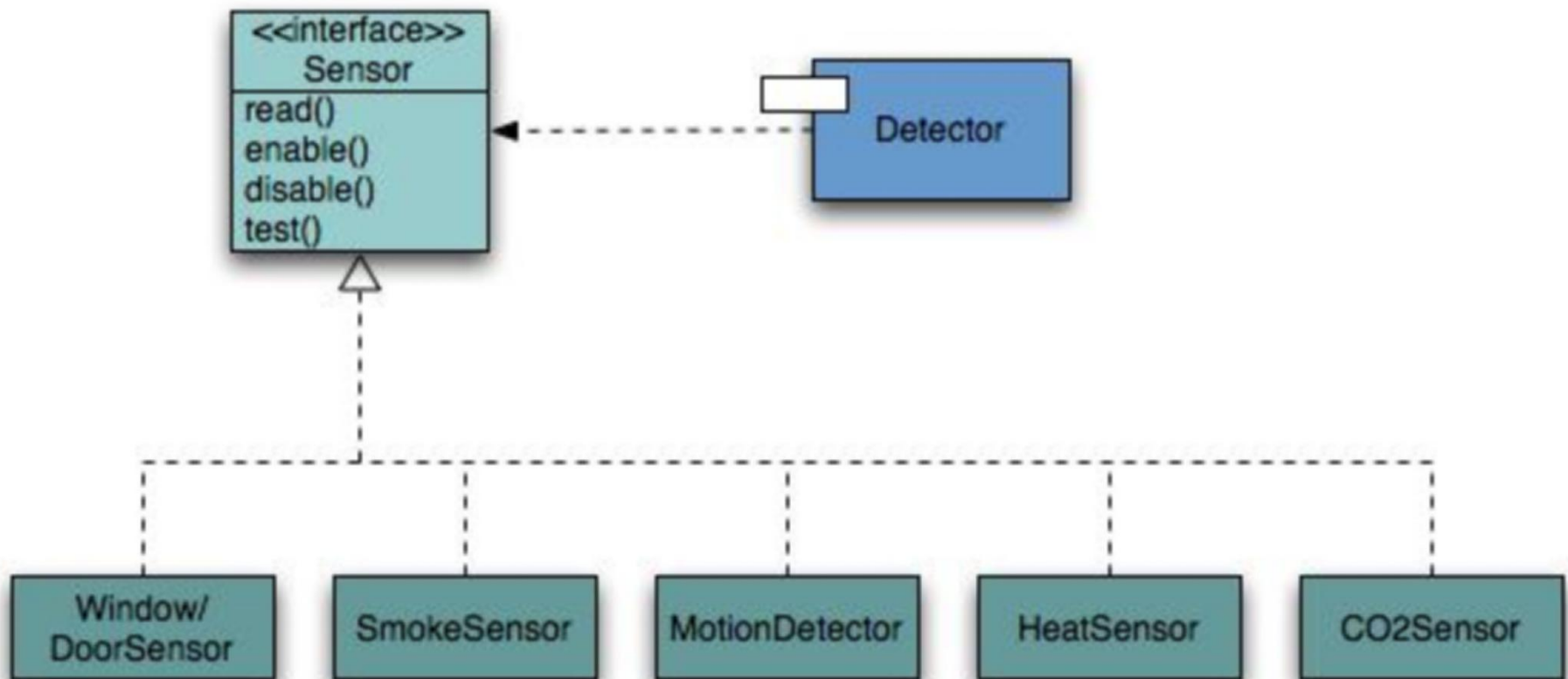
Basic Design Principles

3. 构件的设计原则

1. **开闭原则 (OCP)** ——一个模块 [组件] 应该对扩展开放但对修改关闭
2. **里氏替换原则 (LSP)** ——“子类应该可以替代它们的基类。
3. **依赖倒置原则 (DIP)** ——依赖于抽象。不要依赖实体
4. **接口分离原则 (ISP)** ——许多专用的接口都比一个通用接口好
5. **发布重用等效原则 (REP)** ——重用的粒度就是发布的粒度
6. **共同闭合原则 (CCP)** ——同时修改的类应该放在一起
7. **共同重用原则 (CRP)** ——不能同时复用的类不应该放在一起

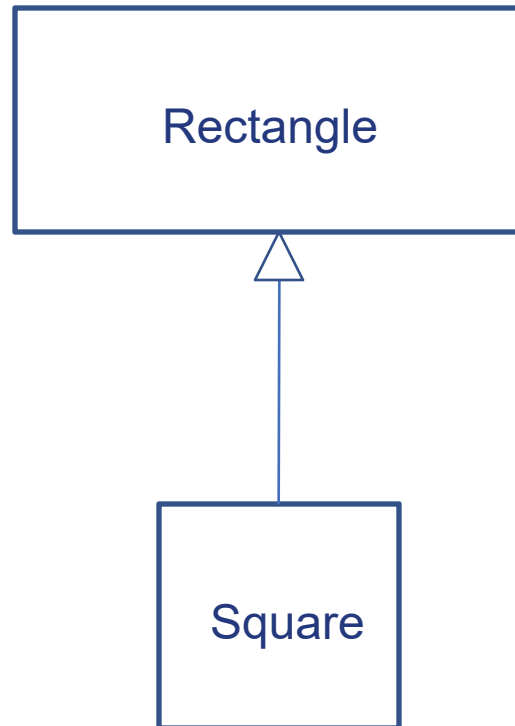
Open-Closed Principle

- ❖ A module should be open for extension but closed for modification.



Substitutability

- ❖ Subclasses should be substitutable for base classes



Is a square a kind of rectangle?

//长方形

```
public class Rectangle{
    private long width;
    private long height;

    public long getWidth() {
        return width;
    }
    public void setWidth(long width) {
        this.width = width;
    }
    public long getHeight() {
        return height;
    }
    public void setHeight(long height) {
        this.height = height;
    }
}
```

//正方形

```
public class Square{
    private long side;

    public long getSide() {
        return side;
    }

    public void setSide(long side) {
        this.side = side;
    }
}
```

//正方形

```
public class Square extends Rectangle{  
    private long side;
```

```
  
    public long getHeight() {  
        return this.getSide();  
    }
```

```
  
    public long getWidth() {  
        return this.getSide();  
    }
```

```
  
    public void setHeight(long height) {  
        this.setSide(height);  
    }
```

```
  
    public void setWidth(long width) {  
        this.setSide(width);  
    }
```

```
        public long getSide() {  
            return side;  
        }
```

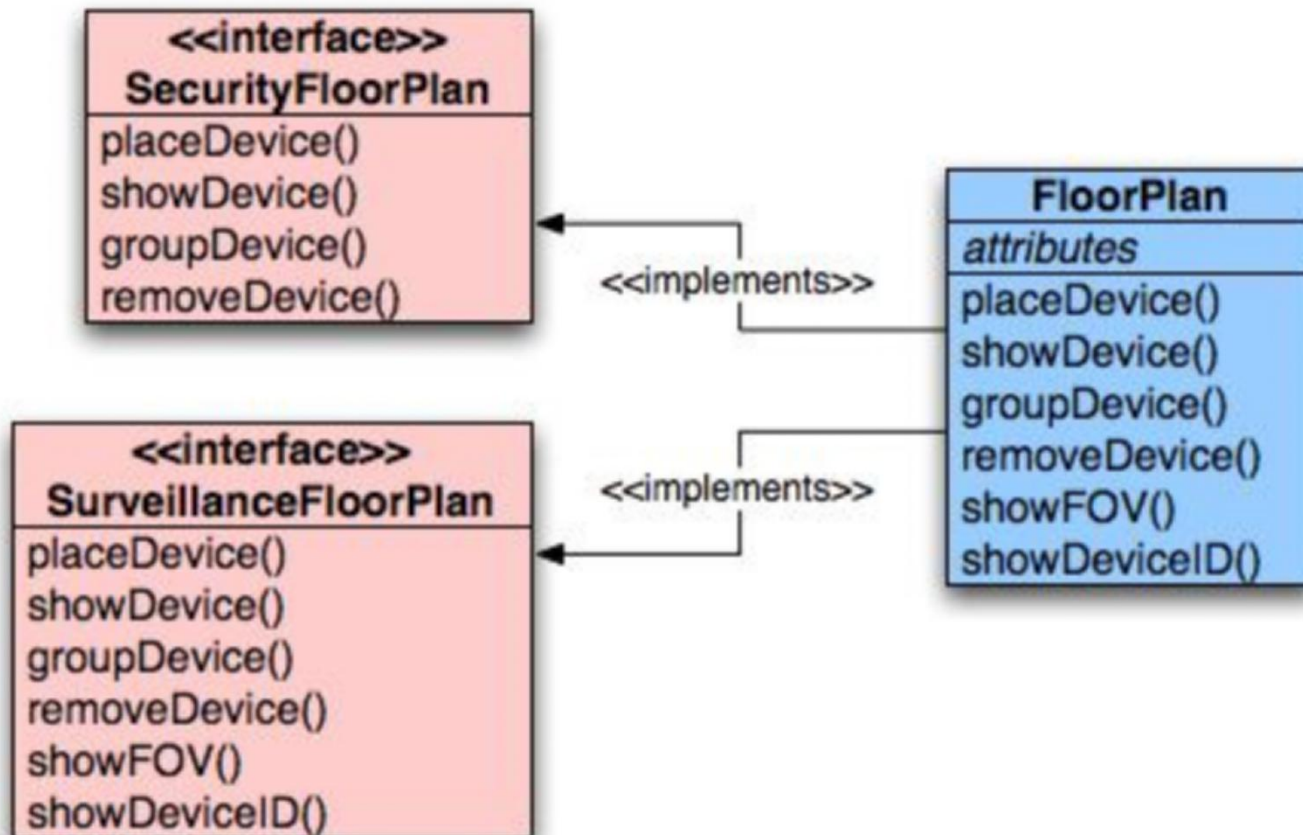
```
  
        public void setSide(long side) {  
            this.side = side;  
        }  
    }
```

```
    public static void resize(Rectangle  
changfangxing) {  
        while(changfangxing.getHeight() <=  
changfangxing.getWidth()) {
```

```
  
        changfangxing.setHeight(changfangxing.g  
etHeight() + 1);  
        test(changfangxing);  
    }  
}
```

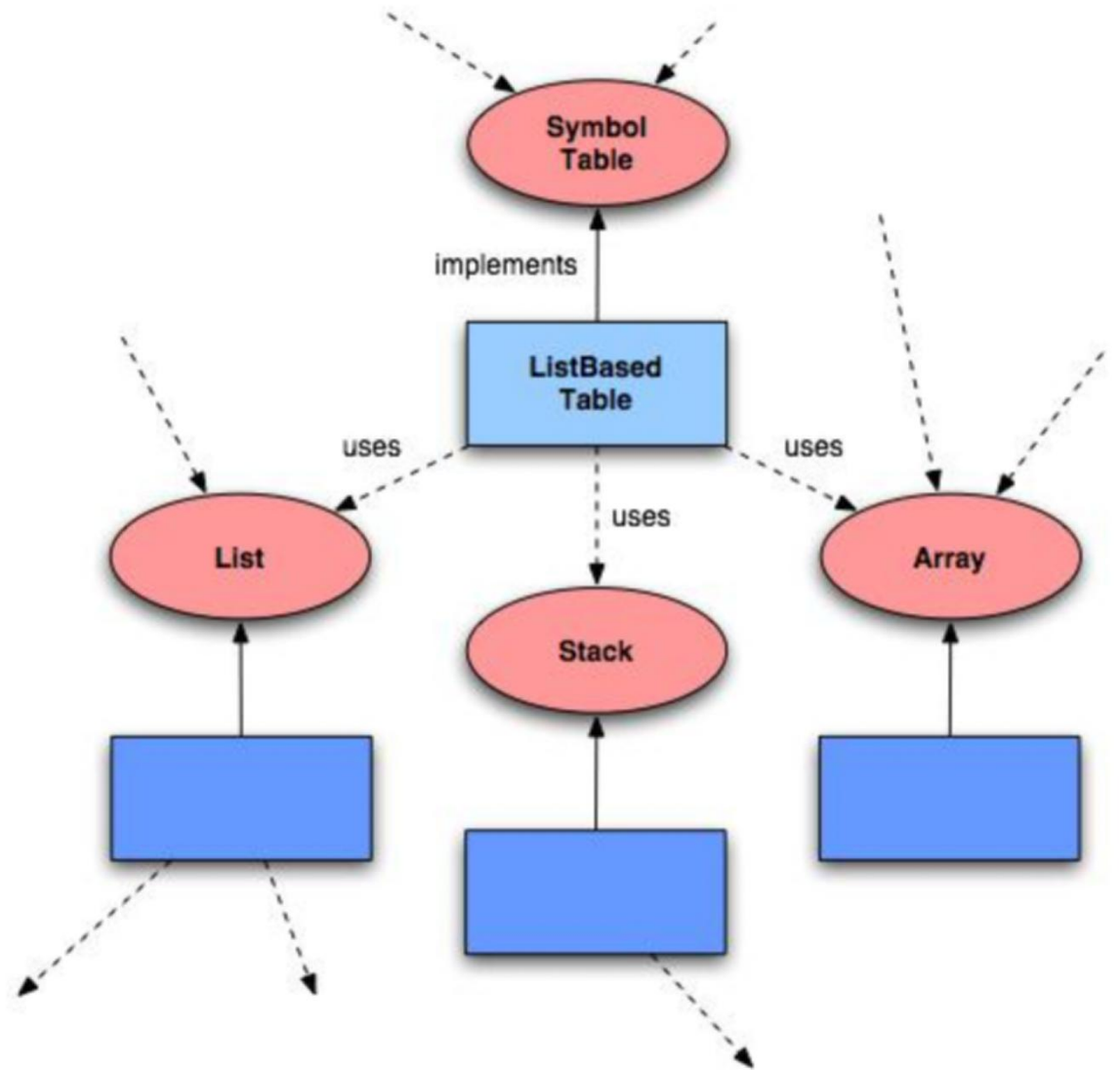
Interface Segregation

- ❖ Many client-specific interfaces are better than one general purpose interface.



Dependency Inversion

- ❖ Depend on abstractions.
Do not depend on concretions.



Basic Design Principles

- ❖ **The Release Reuse Equivalency Principle (REP).** *“The granule of reuse is the granule of release.”*
 - The developer should maintain and evolve the reusable entities.
- ❖ **The Common Closure Principle (CCP).** *“Classes that change together belong together.”*
 - Classes should be packaged cohesively. The developer should maintain and evolve them together.
- ❖ **The Common Reuse Principle (CRP).** *“Classes that aren’t reused together should not be grouped together.”*

Cohesion 内聚性

❖ Conventional view:

- the “single-mindedness” of a module

❖ OO view:

- *cohesion* implies that a component or class encapsulates only attributes and operations that are closely related to one another and to the class or component itself

传统观点:

- 模块的“单一职责”

面向对象观点:

- 内聚性意味着一个组件或类仅封装与彼此密切相关的属性和操作，并且与类或组件自身密切相关。

Cohesion

❖ Levels of cohesion

- **Functional**
- **Layer**
- **Communicational**
- **Sequential**
- **Procedural**
- **Temporal**
- **utility**



High

Low

- 功能性
- 层次性
- 通信性
- 顺序性
- 过程性
- 时间性
- 工具性

Coupling 耦合性

❖ Conventional view:

- The degree to which a component is connected to other components and to the external world

❖ OO view:

- a qualitative measure of the degree to which classes are connected to one another

传统观点:

- 组件与其他组件及外部世界的连接程度

面向对象观点:

- 类之间连接程度的定性度量

Coupling

❖ Level of coupling

- **Content**
- **Common**
- **External**
- **Control**
- **Stamp**
- **Data**
- **Routine call**
- **Type use**
- **Inclusion or import**



High

- 内容耦合
- 公共耦合
- 外部耦合
- 控制耦合
- 印章耦合
- 数据耦合
- 例程调用
- 类型使用
- 包含或导入

Low

Component Level Design-I

- ❖ Step 1. Identify all design classes that correspond to the **problem domain**.
- ❖ Step 2. Identify all design classes that correspond to the **infrastructure domain**.
- ❖ Step 3. Elaborate all design classes that are not acquired as reusable components.
 - Step 3a. Specify **message details** when classes or component collaborate.
 - Step 3b. Identify **appropriate interfaces** for each component.
 - Step 3c. Elaborate **attributes** and define data types and data structures required to implement them.
 - Step 3d. Describe **processing flow** within each operation in detail.

Component-Level Design-II

- ❖ Step 4. Describe **persistent data sources** (databases and files) and identify the classes required to manage them.
- ❖ Step 5. Develop and elaborate **behavioral representations** for a class or component.
- ❖ Step 6. Elaborate **deployment diagrams** to provide additional implementation detail.
- ❖ Step 7. Factor every component-level design representation and always consider alternatives.

组件级设计

4. 进行构件级设计

1. 识别与问题域对应的所有设计类
2. 识别与基础设施领域相对应的所有设计类
3. 细化所有不作为可重用构件的设计类——明确协作信息、接口、属性、操作
4. 描述持久性数据源（数据库和文件）并确定管理它们所需的类
5. 开发并细化类或组件行为表示
6. 细化部署图并提供实现细节
7. 考虑每个组件级设计表示并始终考虑替代方案

1. 确定所有对应于问题域的设计类
2. 确定所有对应于基础设施域的设计类
3. 详细说明所有未作为可重用组件获取的设计类
 - 3a. 在类或组件协作时指定消息细节
 - 3b. 为每个组件确定适当的接口
 - 3c. 详细说明属性并定义实现所需的数据类型和数据结构
 - 3d. 详细描述每个操作中的处理流程
4. 描述持久数据源（数据库和文件）并确定管理它们所需的类
5. 开发并详细说明类或组件的行为表示
6. 详细说明部署图以提供额外的实现细节
7. 分解每个组件级设计表示并始终考虑替代方案